Log Representations and Learning Regimes for Log-based Anomaly Detection: A Replication and Extension Study

Pham Minh Dung Duong (2553085)[1], Thanh Nghiep Quan (2557078)[1], Tran Vo (2553737)[1]
[1] *Dep. of Computer and Software Engineering, Polytechnique Montréal, Canada*

***Abstract*—Machine learning models for log-based anomaly detection need the raw logs to be parsed and turned into numbers. Wu et al. [1] studied how this log representation step affects the models. We replicate their first research question for two supervised models on two datasets. We also check whether removing correlated features helps. We then extend the study on one dataset with an unsupervised and a semi-supervised model, a statistical ranking, and an explanation of the best model. We parse HDFS and BGL with Drain and the preprocessing of Qin et al. We train a Random Forest on message count vectors and an LSTM on BERT template embeddings, with and without correlation and redundancy analysis. On HDFS we add PCA (unsupervised) and LogGPT (a GPT-2 model trained on normal sessions only). We retrain the four models on 10 random train/test splits, rank them with the Scott-Knott ESD test, and explain the best model with SHAP. The Random Forest results are close to the paper (F1 0.999 in both on HDFS, 0.919 against 0.891 on BGL). The LSTM is better than reported on HDFS (0.987 against 0.959), where we trace the recall gap to a windowing bug in the original code, and worse on BGL (0.870 against 0.931). Removing correlated features does not reliably improve accuracy. On HDFS, the ranking is RF $>$ LSTM $>$ LogGPT $>$ PCA for both F1 and AUC. LogGPT reaches a mean F1 of 0.856 from 5,000 normal sessions, and its F1 depends on its Top-K threshold. SHAP shows that the Random Forest relies mostly on one error template and on the number of events of the block write pipeline. On HDFS, a supervised model on simple count features gives the best results. Without labels, the threshold and the training protocol matter as much as the model.**

I. Introduction

Software systems write logs that record what happens at run time. When a failure occurs, engineers often look at these logs first. Because modern systems produce millions of log lines, automated log-based anomaly detection has been studied widely [2], [3]. Most approaches follow the same pipeline: the raw log lines are parsed into event templates, grouped into sequences (sessions), represented as numeric features, and given to a model that labels each session as normal or anomalous.

Wu et al. [1] studied the representation step of this pipeline. In their RQ1 they compared six representations with seven models on four datasets. They found that the choice of representation has a non-negligible effect on the results, that the simple Message Count Vector (MCV) performs best overall, and that BERT embeddings work best for models that use the order of events. This report presents a partial replication and an extension of that work. It answers four research questions:

Table I: Datasets after parsing and partitioning

	Templates	Sessions	Anomalous	Events/sess.	Train	Test
HDFS	47	575,061	16,838 (2.9%)	19.4	402,542	172,519
BGL	421	826	409 (49.5%)	5,748.1	660	166

- **RQ1 (Replication):** Can we reproduce the RQ1 results of Wu et al. for a classical and a deep learning supervised model on HDFS and BGL?
- **RQ2 (Feature Selection):** Does removing correlated and redundant features change the performance of the classical model?
- **RQ3 (Extension):** How do an unsupervised and a semi-supervised model compare with the two supervised models when all four are trained and tested under the same protocol?
- **RQ4 (Explanation):** Which log events drive the decisions of the best model, and are these explanations stable?

Our replication found three issues in the original replication package that change the reported numbers. We fixed them and measured the effect of each one. Our replication package is available at https://github.com/nghiepqt/devops_replicate. It contains the code, configuration files and result files behind the numbers in this report (folders `part1/` and `part2/`), and every design decision is logged there with its reason (`DECISIONS.md`).

II. Approach

Figure 1 gives an overview. Part 1 (RQ1, RQ2) uses HDFS and BGL. Part 2 (RQ3, RQ4) uses HDFS only.

A. Datasets

We use two of the four datasets of the original study, both from Loghub [4] (table I). **HDFS** [5] comes from a Hadoop file system. Each log line refers to a data block, and each block is labelled normal or anomalous by the dataset authors. **BGL** [6] comes from a BlueGene/L supercomputer, where each line has its own label (alert or not). We chose these two because they are widely used in the literature and they differ in a useful way. HDFS has many short sessions with few anomalies, while BGL has long windows and many anomalies. The line counts match the original study (11,175,629 and 4,747,963 lines).

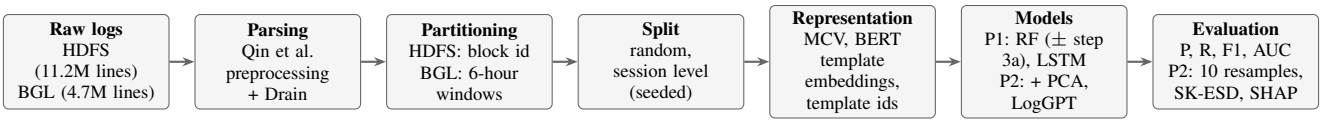


Figure 1: Overview of our pipeline. P1 = Part 1 (replication, HDFS and BGL); P2 = Part 2 (extension, HDFS).

B. Log parsing

We parse the logs with Drain [7] (depth 4, similarity threshold 0.5, at most 100 children), the parser used by the original study. Before parsing, we apply the preprocessing framework of Qin et al. [8], as suggested by the assignment. It masks variable parts (IP addresses, paths, block ids, numbers) with 14 ordered regular expressions. It first checks which expressions match the first 2,000 lines and then applies only those to the whole file. We reused the authors' code without changes. Block ids (HDFS) and labels and timestamps (BGL) are extracted before masking. We obtain 47 templates for HDFS (the original study reports 46) and 421 for BGL.

To check the parsing quality, we ran the same pipeline on the Loghub-2.0 samples of 2,000 lines with corrected ground truth [9]. Grouping accuracy was 1.000 on HDFS and 0.970 on BGL. This is equal to or higher than Drain with the regular expressions used by the original study (0.998 and 0.970). Grouping accuracy matters most for count features, because two lines in the same group are counted as the same event.

C. Partitioning and splitting

For HDFS, one session contains all lines that mention the same block id, in log order. For BGL, we use fixed, non-overlapping 6-hour windows, as in the original study. A window is anomalous if at least one of its lines is labelled as an alert. This gives 826 windows, while the original paper reports 718. We tried several variants (aligning windows to the epoch, using another time field, removing small windows), but none gives 718. The original code does not include the windowing step, so we kept the literal definition of the paper. As in the paper, we split the sessions at random (70/30 for HDFS, 80/20 for BGL) with a fixed seed. The main runs use seed 42. To measure the variance, we also repeat Part 1 with seeds 0 to 4.

D. Representations

Message Count Vector (MCV). One feature per template counts how many times it appears in the session. The vocabulary is built from the training sessions only, and events of templates never seen in training are dropped.

BERT embeddings. Each template text is given to the pre-trained `bert-base-uncased` model [10]. Its embedding is the mean of the token vectors of the second-to-last layer (768 dimensions, at most 50 tokens). These settings follow the original replication package. A session becomes a sequence of these vectors, which the LSTM reads in order.

Why these pairings. We pair the level of each representation with the mechanism of the model, as Wu et al. do: "all of our studied traditional models take sequence-level representation as input", while "LSTM models require fixed length event-level representation as input" [1] (§4.3.1). For the RF we use MCV because it is exactly the fixed-dimension, order-free feature vector that a forest of CART trees consumes [11], and because the original study ranks it first: in their Scott-Knott ESD ranking, the three classical representations "outperformed their semantic-based counterparts, with MCV achieving the best rank" [1] (§5.1). For the LSTM we use BERT embeddings because each of its ordered input positions needs a dense, context-aware vector, and because BERT "achieves the best performance for 7 out of the 8 cases of the models that can leverage sequential information" [1] (§5.1). PCA keeps the event count matrix it was designed for [5]. LogGPT, finally, reads the raw template-id sequence, because its GPT-2 backbone learns its own representation during training [12].

E. Supervised models (Part 1)

Random Forest (RF). A classical model [11], robust to irrelevant features and fast to train. We use the settings of the replication package: 10 trees, $\sqrt{p}$ features per split, no depth limit, no class weights. The anomaly score is the predicted probability of the anomalous class.

LSTM. A deep learning model [13] that can use the order of events. We use the architecture of the package: one LSTM layer with 8 hidden units, a linear output layer, cross-entropy loss, Adam with learning rate 10^{-3}, 30 epochs, and balanced sampling of the sessions. Sessions are cut into sliding windows (size 30, step 1 for HDFS; size 50, step 50 for BGL). Every window gets the label of its session, and the session score is the highest anomaly probability of its windows.

Changes to the replication package. We found three issues in the package and fixed them: (1) For RF, the metric function is called with its arguments swapped, so the printed precision is the recall and vice versa. (2) For many sessions shorter than the window, the package keeps only the last $w - L - 1$ events of a session of length L. With $w = 30$, this affects sessions of 15 to 28 events, and a session of 19 events (the HDFS median) keeps 10 events. We feed whole sessions and use packed sequences, so padding never reaches the last hidden state. (3) The package keeps the epoch with the best F1 on the test set. We hold out 10% of the training sessions and select the epoch on this validation set. We also train in mini-batches of 512 windows instead of one optimizer step per session, because the latter would take several hours per run on HDFS. To measure the effect of these changes, we ran ablations that reproduce the package's windowing and training (section III-A).

F. Correlation and redundancy analysis (step 3a)

Correlated features can split the importance of one underlying event across several features and make a model harder to interpret. We apply the analysis to the RF features, using the training data only. First, we compute the Spearman correlation between all features and cluster them hierarchically (average linkage, distance $1 - |\rho|$). We cut the tree at 0.3 ($|\rho| \geq 0.7$) and keep, in each cluster, the template that occurs most often. Second, we repeatedly remove the feature with the highest variance inflation factor (VIF) until all VIFs are at most 5. We do not apply this step to the LSTM. Its 768 inputs are embedding dimensions, not events, and the embedding matrix of the 47 HDFS templates has rank 47. Most dimensions are therefore exact linear combinations of others, so every VIF would be infinite.

G. Unsupervised and semi-supervised models (Part 2)

Labelled anomalies are rare and costly in practice, so we add two models that do not need them. They come from two different categories, as required.

PCA (classical, unsupervised). PCA [5] is the classical baseline for count features. We port the implementation of loglizer [2] and keep its defaults. The MCV is weighted with TF-IDF and centred, both fitted on the training data. We keep the principal components that explain 95% of the variance. The anomaly score of a session is its squared prediction error (SPE), i.e., the squared length of its projection on the remaining (residual) components. A session is anomalous if the SPE is above the Q-statistic threshold at $\alpha = 0.001$ [14]. PCA is trained on the whole training split with the labels removed, so about 2.9% of its training sessions are anomalous.

LogGPT (LLM-based, semi-supervised). LogGPT [12] trains a small GPT-2 model [15] from scratch (6 layers, 6 heads, embedding size 60) to predict the next template id of normal sessions. During testing, a session is anomalous if, at any position, the actual next template is not among the K most likely predictions. A template never seen in training counts as a miss. We chose LogGPT because it represents the LLM-based category, its code is public, and it is small enough to train on a laptop GPU. It has about 0.3 million parameters, about 420 times fewer than the smallest public GPT-2 model (124 million), and a vocabulary of 20 to 22 tokens: the templates of its training sessions and five special tokens. It uses no pre-trained weights, no text prompts and no external API. As in the paper, it is trained on 5,000 normal sessions sampled from the training split (labels are used only for this filter): 4,500 for training and 500 for validation. Pre-training runs for 100 epochs (learning rate 10^{-4}, batch size 16). It is followed by reinforcement learning (RL) fine-tuning with a Top-K reward: $+1$ when the actual next template is in the Top-K, -1 otherwise (learning rate 10^{-6}, at most 20 episodes, early stopping on the validation loss). K is 50% of the unique templates of the training sessions, rounded down. In each RL step, the model generates one sequence with beam sampling (20 beams, sampling enabled). Neither the original code nor the paper sets a temperature, so the default of the library applies (1.0). Scoring uses one forward pass in evaluation mode, without sampling, so the temperature can affect the results only through RL training.

We ported the authors' code to current PyTorch and fed it the same template sequences that the LSTM sees, instead of using its own parser. Where the code and the paper disagree, we followed the code. The paper describes the RL step as PPO [16], but the code makes one policy-gradient update per generated sequence, without the PPO ratio and clipping. The code also uses the last pre-training epoch, not the one with the best validation loss (on our runs the difference in validation loss is at most 1.2%). We removed one step of the code that evaluated the test set after each RL episode. Its output was only printed, but using test data during training breaks our protocol. To verify the port, we compared its predictions with the original prediction function on about 350 test sessions per resample, and they were identical in all cases.

H. Evaluation protocol

We report precision (P), recall (R), F1, and ROC-AUC at the session level, with the anomalous class as positive. Accuracy is not used because 97% of the HDFS sessions are normal. F1 evaluates the decisions that a user would see, while AUC evaluates the ranking of sessions without a threshold. To compute AUC, every model outputs a continuous score, and its decision is a threshold on that same score. RF uses the probability (≥ 0.5) and the LSTM the highest window probability (≥ 0.5). PCA uses the SPE ($>$ Q-statistic). For LogGPT, the score is the highest rank of the actual next template over all positions (rank 1 = most likely), and the decision is score $> K$.

In Part 2, the comparison is only meaningful if everything except the model is the same. For each of 10 resamples (seeds 0 to 9), we draw a new random 70/30 split with the Part 1 split function and retrain all four models. All models are scored on the full test split of that resample. We check in the code that the session ids are identical. No model uses test data or anomaly labels to choose epochs, thresholds or K. We also record training and inference time on the same machine (Intel i5-13500HX, NVIDIA RTX 4060 Laptop GPU), with one run at a time.

I. Statistical ranking

We rank the models with the Scott-Knott ESD test [17] (R package version 3.0.0 [18]), once on F1 and once on AUC, with the 10 resamples as observations. The original study also used this test to rank representations. Unlike pairwise tests, Scott-Knott ESD splits the models into groups that do not overlap, and it merges groups whose difference has a negligible effect size. We use the non-parametric version because 10 values per model are too few to check normality, and RF values are close to the upper bound of 1. This version orders the models by median, splits them where the Kruskal-Wallis statistic is highest, and keeps two groups apart only if Cliff's δ [19] between them is not negligible ($|\delta| \geq 0.147$).

Table II: RQ1: Our results (seed 42) compared with Wu et al. [1]. The last column is the mean F1 over seeds 0–4 and 42 (sample std).

		Paper			Ours, seed 42				
Data	Model	P	R	F1	P	R	F1	AUC	F1 (6 seeds)
HDFS	RF+MCV	.998	1.000	.999	.998	.999	.999	1.000	.998 ± .000
HDFS	LSTM+BERT	.998	.923	.959	.976	1.000	.987	1.000	.976 ± .008
BGL	RF+MCV	.830	.963	.891	.963	.878	.919	.980	.922 ± .019
BGL	LSTM+BERT	.871	1.000	.931	.769	1.000	.870	.994	.828 ± .033

J. Explanation

We explain the best model of RQ3 with SHAP [20]. For tree models, the TreeExplainer computes exact SHAP values in a short time [21]. SHAP gives one additive contribution per feature and per session, which supports both global and local analysis. We explain the anomaly probability of the RF of seed 0 on all 5,007 anomalous test sessions and 5,000 normal test sessions sampled at random. Because SHAP values are additive, we also sum them over the correlation clusters of step 3a. This gives the importance of a group of correlated templates without removing any feature from the model. We cross-check the grouped ranking with two other rankings: the SHAP values of the RF trained with step 3a, and the permutation importance on F1. Finally, we repeat the grouped analysis for all 10 resamples to measure stability.

III. Results

A. RQ1: Replication of the supervised models

Table II compares our results with the paper.

HDFS, RF. The result is reproduced (F1 0.999 in both). HDFS is almost separable with event counts, as RQ4 will show, so small differences in parsing or settings do not matter.

HDFS, LSTM. Our F1 is higher than the paper (0.987 against 0.959), and the gap comes from recall (1.000 against 0.923). We looked for the cause with an ablation. When we only switch to the package's windowing (short sessions truncated, right padding, output read at the last padded step), recall drops to 0.938, close to the paper's recall, and F1 drops to 0.876. This explains the recall gap but not the whole F1 gap, because the precision of this ablation (0.822) is below the paper's (0.998). The package also differs in other ways (one optimizer step per session, window-level evaluation), but we could not test them separately on HDFS. Epoch selection is not the cause: the best test F1 over all 30 epochs is 0.988, against 0.987 for the epoch chosen on validation.

BGL, RF. Our F1 is a little higher than the paper (0.919 against 0.891), and this holds for all six seeds (0.922 ± 0.019, lowest value 0.899). Our precision and recall look swapped compared with the paper (0.963/0.878 against 0.830/0.963). This matches issue (1) in the package: the paper's precision is in fact the recall. F1 is not affected because it is symmetric in P and R.

BGL, LSTM. This is the only result below the paper. Our F1 is 0.870 (0.828 ± 0.033 over 6 seeds) against 0.931. Recall is the same (1.000), so the whole gap is in precision: at seed 42, 27 of the 76 normal test windows are flagged. We checked three possible causes on our side. First, an ablation that follows the package's windowing, padding and one-step-per-session training gives F1 0.853, so our changes do not explain the gap. Second, the epoch we select on validation is also the best one on the test set (0.870). Third, a rerun gives the same training curve. The remaining explanations come from the data. We have 826 windows instead of 718, so the window boundaries and labels differ. The test set has only 166 windows, so one false positive changes F1 by about 0.004, and F1 varies between 0.785 and 0.870 across seeds. Also, each LSTM window inherits the label of its 6-hour session, which gives noisy labels to an LSTM with only 8 hidden units: its training loss stays between 0.36 and 0.60.

We also ran the HDFS models with the split that the package actually uses. It is not a random shuffle but the loglizer "uniform" split, which puts the first 70% of each class in the training set. Both models score higher with it (F1 1.000 for RF, 0.995 for the LSTM), so the split does not explain any gap with the paper.

Answer to RQ1: The RF results are reproduced on HDFS and slightly higher than the paper on BGL. The LSTM differs from the paper in both directions. On HDFS, the recall gap is explained by a windowing bug in the original code. On BGL, it is most likely due to the different number of windows and the small, noisy test set.

Table III: RQ2: RF+MCV without and with step 3a (F1: mean ± std over 6 seeds; features and time: seed 42)

	Features		F1		Train time (s)	
Data	w/o	with	w/o	with	w/o	with
HDFS	47	28	.998 ± .0003	.996 ± .0006	0.5	117.7
BGL	410	119	.922 ± .019	.929 ± .041	0.1	73.0

B. RQ2: Effect of correlation and redundancy analysis

On HDFS, the correlation analysis finds 31 clusters (16 features removed) and the VIF step removes 3 more. On BGL, 410 features become 152 and then 119 (table III). On HDFS, F1 decreases a little but in all six seeds (false positives go from 9 to 21 at seed 42). The removed templates are interchangeable on average but not in every session. One example is the two `allocateBlock` templates ($\rho = -1.0$): every block has exactly one of the two. A Random Forest is not harmed by correlated features when it predicts, because each split uses one feature. Removing them can therefore only remove information. On BGL, the mean F1 increases by 0.007, which is less than one standard deviation, and the standard deviation doubles. With 410 features and 660 sessions, removing redundant templates helps the RF sample useful features more often. However, keeping the most frequent template of a cluster sometimes drops a rarer template that is specific to anomalies (seed 2 falls to 0.848).

Answer to RQ2: Removing correlated and redundant features does not improve the RF in a reliable way. It slightly lowers F1 on HDFS and increases the variance on BGL. Its value is for interpretation, which we use in RQ4.

C. RQ3: Supervised, unsupervised and semi-supervised models

Table IV and fig. 2 summarize the 10 resamples. As a regression check, the RF and LSTM results of seeds 0 to 4 are identical to those of Part 1.

Ranking. Scott-Knott ESD places each model in its own group, in the same order for F1 and AUC: RF, LSTM, LogGPT, PCA. For F1, the groups are well separated: the distributions do not overlap and every pairwise Cliff's δ is ± 1. For AUC, RF and LSTM are also in different groups ($\delta = 0.54$), although the median difference is only about 0.0001. Cliff's δ only looks at the order of the values, not at the size of the difference. So the test tells us that RF has the higher AUC in most resamples, but not that the difference matters in practice. RF, LSTM and LogGPT all have a median AUC above 0.99 (the lowest LogGPT value is 0.982), so their differences in F1 come mostly from where each model puts its threshold.

Supervised models. RF has the best precision, F1 and AUC. The LSTM has about the same recall (0.999, slightly higher than RF in 8 of 10 resamples) but about 16 times more false positives (217 against 13 per resample). Two choices from the original design may explain this. Each window gets the label of its whole session, so many normal-looking windows of anomalous sessions are labelled anomalous. In addition, the session score is the maximum over windows, so one uncertain window is enough to flag a session.

PCA. PCA has a high precision (0.971) but finds only 64% of the anomalies. Its five principal components explain 96% of the variance and are dominated by the templates whose counts vary most: `Served block` and `Got exception while serving` (read operations), the four replication templates, `Verification succeeded`, and `Receiving empty packet`. A session that deviates along these directions is treated as normal variation. This is why 44% of the missed anomalies (mean over the resamples) contain the replication request template. It is also why the `Receiving empty packet` template, which appears only in anomalous sessions, ended up in the normal subspace: its anomalous training sessions were enough to give it a large variance. On the other hand, all anomalies that contain the deletion error template (T33, about 30% of the anomalies) are detected, because this template lies in the residual subspace. PCA is also stable (F1 std 0.003). It has no random initialization, and two training splits share about 70% of their sessions, so the ten models are almost the same. They also make the same errors. In all 45 pairs of resamples, the anomalies missed are identical among the anomalous sessions that the two test splits share (on average 561 of 1,538 are missed by both). The missed sessions look like normal ones: their median SPE is 0.47, below the 99th percentile of the normal sessions (0.55) and far below the threshold (about 3.9). A better threshold would help little: the best threshold, chosen on the test set as a diagnostic, gives a mean F1 of 0.796 against 0.768. So the low F1 comes from what the normal subspace contains, not from the threshold or from noise.

The training protocol also has a large effect on PCA. With 5,000 random sessions, F1 is 0.662 ± 0.221 (table VI), because the number of components (3 to 6) and the threshold (0.2 to 18.7) change with the sample. When PCA is trained on the same 5,000 normal sessions as LogGPT, the mean F1 drops from 0.768 to 0.487. The vocabulary then contains only the 15 to 17 templates seen in normal sessions, so the templates that only occur in anomalies are not features at all. The LogGPT paper reports F1 0.087 for PCA under its own normal-only protocol.

LogGPT. LogGPT is trained on 80 times fewer sessions and never sees an anomaly, but it reaches a median F1 of 0.845 and an AUC of 0.998. One mechanism explains a large part of its recall. On average, 1,797 of the 5,071 anomalous test sessions (35%) contain a template that never appears in the 5,000 normal training sessions, and only 17 normal test sessions do. These sessions are almost always flagged (99.9%). The other anomalies must be detected from the order of events, and this is where recall varies (between 0.678 and 0.991). The spread of F1 across resamples is also related to K. The 50% rule gives $K = 7$ when the training sample contains 15 unique templates and $K = 8$ when it contains 16 or 17, which happens in 7 of the 10 resamples. Since we store the scores, we can apply another K without retraining (table V). With $K = 7$ for all resamples, the mean F1 is 0.892 (P 0.880, R 0.910), close to the 0.901 reported in the paper. This is only a sensitivity analysis and is not used in the ranking, because choosing K on test results would break our protocol. Even with $K = 7$, every LogGPT F1 value (0.826–0.935) stays below every LSTM value and above every PCA value, so the ranking would not change.

The threshold explains only part of the spread: with K fixed at 7, the standard deviation of F1 goes from 0.059 to 0.043. The rest comes from the training runs. Each resample uses another sample of 5,000 sessions and another initialization, and training also uses dropout and, in RL, sampled sequences. Most of the spread is in recall (std 0.105, against 0.033 for precision). Our seeds change all of these sources together. To separate part of them, we repeated pre-training (without RL) five times on the training sample of seed 0 and changed only the random seed. F1 then has a standard deviation of 0.010 (0.830–0.857), against 0.039 over the 10 resamples without RL, and recall 0.010 against 0.058. So without RL, most of the spread seems to come from the choice of training sessions and not from the initialization or dropout. This is one sample and five runs, so it is only an indication. We did not repeat this test with RL (about 53 minutes per run) and did not vary the temperature, so we do not know how much the sampling in RL adds.

The RL fine-tuning moves LogGPT in the direction reported by the paper: precision decreases in all 10 resamples, recall

Table IV: RQ3: Performance over 10 resamples of HDFS (same test sets for all models). P, R and FP/FN: mean; F1 and AUC: median [IQR]; SK = Scott-Knott ESD group (1 = best); time: median per resample.

Model	Labels used (train sessions)	P	R	F1	SK F1	AUC	SK AUC	FP / FN	Train (s)	Infer (s)
RF+MCV	all (402,542)	.997	.999	.9982 [.9979–.9983]	1	.9999 [.9999–1.0000]	1	13 / 6	0.4	0.07
LSTM+BERT	all (402,542)	.959	.999	.9785 [.9748–.9835]	2	.9999 [.9997–.9999]	2	217 / 4	78.5	0.58
LogGPT	normal only (5,000)	.889	.831	.8451 [.8144–.9086]	3	.9985 [.9950–.9987]	3	533 / 856	3,399.7	0.53
PCA	none (402,542)	.971	.636	.7688 [.7665–.7698]	4	.9589 [.9580–.9606]	4	98 / 1,847	0.3	0.10

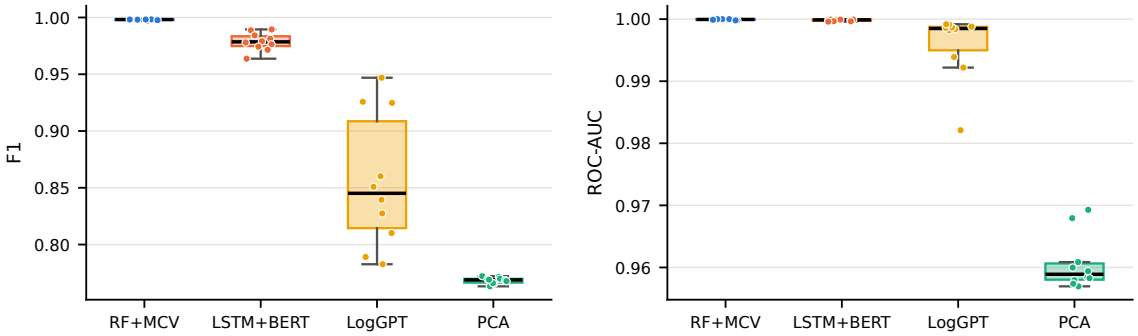


Figure 2: F1 and ROC-AUC of the four models over the 10 resamples of HDFS (dots are resamples).

Table V: LogGPT over 10 resamples (mean ± std) and the values reported by Han et al. [12]. "$K = 7$" re-thresholds the stored scores (sensitivity only).

Variant	P	R	F1	AUC
w/o RL, K = 50% rule	.941 ± .012	.772 ± .058	.848 ± .039	.964
with RL, K = 50% rule	.889 ± .033	.831 ± .105	.856 ± .059	.996
with RL, $K = 7$	.880	.910	.892 ± .043	–
Paper, w/o RL	.932 ± .015	.790 ± .101	.853 ± .065	–
Paper, with RL	.884 ± .030	.921 ± .066	.901 ± .036	–

Table VI: Control for training size: F1 (mean ± std over 10 resamples, same test splits as table IV). Not used in the ranking. In parentheses: mean anomalous sessions in the sample; "all" is the whole training split (402,542).

Sessions	500 (13)	5,000 (147)	50,000 (1,479)	all (11,767)
RF, labels	.911 ± .043	.985 ± .008	.996 ± .001	.998 ± .000
LSTM, labels	.313 ± .316†	.874 ± .064	.957 ± .028	.979 ± .008
PCA, no labels	.709 ± .091	.662 ± .221	.770 ± .008	.768 ± .003
LogGPT, normal	–	.856 ± .059	–	–
PCA, normal	–	.487 ± .094	–	–

† Not reliable: 3 of 10 validation sets contain no anomaly.

increases in 6, and AUC increases in all 10 (from 0.964 to 0.996). However, the mean F1 gain is only 0.008, against 0.048 in the paper. The Top-K reward pushes the model to give a higher probability to the actual next template, so ranks become more informative (higher AUC). But at a fixed K the effect on decisions is mixed. RL also makes F1 more sensitive to K: the mean F1 at $K = 7$ and $K = 8$ differs by 0.051 with RL and by less than 0.001 without it.

Cost. RF and PCA train in less than one second. The LSTM needs about 80 seconds, and LogGPT about 57 minutes per resample (4 minutes of pre-training and 7 to 11 RL episodes of about 5.5 minutes). The RL step is slow because it generates one sequence with beam sampling for each training session, one session at a time.

Control for training size. The models above use different training sizes and different label information, so the ranking does not compare algorithms alone. To check how much this matters, we retrained RF, the LSTM and PCA on n random sessions of each training split and scored them on the same test splits (table VI). These samples keep the natural class mix, so the supervised models still see anomaly labels. LogGPT and PCA can be compared directly, because both can use the same 5,000 normal sessions: LogGPT has the higher F1 and AUC in all 10 resamples. At 5,000 sessions, the LSTM reaches a mean F1 of 0.874 against 0.856 for LogGPT. It is higher in only 6 of 10 resamples, and its AUC is lower in 8 of 10. So much of the distance between the two in table IV (0.979 against 0.856) comes from training size. The RF stays ahead with 5,000 sessions (F1 0.985, higher than LogGPT in all 10 resamples). With only 500 sessions, which contain about 13 anomalies, its F1 of 0.911 is still higher in 8 of 10 resamples. Its AUC of 0.960 is lower than that of LogGPT in 9 of 10, however, so LogGPT ranks sessions better at this size. We did not train LogGPT on more sessions, because one run with RL already takes about 57 minutes. The LogGPT paper reports that its F1 on HDFS changes little with the training size [12], but we did not check this.

Answer to RQ3: With labels, the RF is the best and cheapest model, followed by the LSTM. Without anomaly labels, LogGPT is the better of the two new models, but its F1 depends on the threshold K. PCA misses about a third of the anomalies, always the same ones, and its results depend

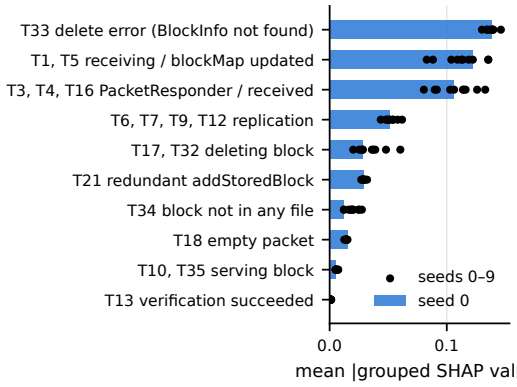


Figure 3: Importance of the top-10 correlation clusters for RF+MCV (bars: seed 0; dots: the 10 resamples, with the seed-0 clusters).

Table VII: RQ4: Agreement between importance rankings (seed 0; τ = Kendall's τ, Top-5 = overlap of the top-5 lists)

Comparison	Items	τ	Top-5
Grouped SHAP vs SHAP of RF + step 3a	27	0.74	4/5
Grouped SHAP vs grouped permutation	30	0.75	5/5
SHAP vs permutation, per feature	46	0.49	2/5

on the training protocol. With the same 5,000 sessions, the RF stays first, the LSTM and LogGPT have close F1 values, and PCA is last.

D. RQ4: Explaining the Random Forest

Global explanation. Figure 3 shows the grouped importance. The most important single template is T33, `Unexpected error trying to delete block. BlockInfo not found in volumeMap`. It appears in 31% of the anomalous test sessions and in 0.04% of the normal training sessions. The next two clusters are the block write pipeline: `Receiving block`, `blockMap updated`, `PacketResponder`, and `Received block`. A normal block is written to three replicas, so these templates usually appear 3 times. Anomalous blocks have fewer of them on average (1.9 `Received block` events against 3.0), which indicates a write that did not complete. Then come the replication cluster and `Redundant addStoredBlock request`. The model therefore uses two kinds of evidence: explicit error messages, and missing or extra events in a normal workflow.

Agreement. The grouped ranking agrees well with the RF trained with step 3a and with the grouped permutation importance (table VII). Per feature, SHAP and permutation importance agree less ($\tau = 0.49$). When one feature of a correlated pair is permuted, the model can still use the other one, so the F1 barely changes. Permutation importance therefore underrates correlated templates such as the write pipeline. This supports grouping the features before interpreting them.

Local explanations. At seed 0, the RF makes 9 false negatives and 12 false positives among 172,519 test sessions. Four of the false negatives contain T33, but also 8 to 10 `Served block` and 4 `Got exception while serving` events. These read events are common in normal blocks and push the score below 0.5. The five false positives with the highest scores all contain a replication request and four (instead of three) `blockMap updated` and `Deleting block` events. These normal blocks have the extra replica events that are typical of anomalous blocks. In both cases, the errors come from sessions that mix normal and anomalous patterns, not from a single misleading template.

Stability. With the seed-0 clusters, T33, the two write-pipeline clusters and the replication cluster are in the top 5 in all 10 resamples. However, the clusters themselves change when step 3a is recomputed on another training split (adjusted Rand index 0.61 to 0.93 with seed 0). In 7 of the 9 other seeds, templates 1, 3, 4 and 5 form a single cluster, which is then the most important one. The order between the error template and the write pipeline therefore depends on how the clusters are cut. The set of important events does not.

Answer to RQ4: The RF relies on a small and stable set of events: one deletion error and the counts of the write and replication workflows. Grouping correlated templates gives explanations that agree with other methods, but the grouping itself should be checked across resamples.

IV. Threats to Validity

Internal validity. We ported LogGPT and PCA to current libraries and checked both against the original code. GPU training is not always bit-for-bit reproducible. In Part 1, a rerun of the LSTM gave the same training curve, but a rerun of LogGPT pre-training with the same seed was close and not identical (F1 0.832 and 0.830, AUC 0.959 and 0.947), so small differences between LogGPT runs may not be meaningful. The BGL windows (826 against 718) differ from the paper, which limits the comparison for BGL.

Construct validity. In RQ3, the models use different amounts of label information (all labels, normal-only, none) and different training sizes (5,000 against 402,542 sessions). The control for training size (table VI) measures the effect of the training size, but it does not remove the threat. At equal size, the supervised models still use anomaly labels, LogGPT was not trained on more sessions, and these runs are not ranked. So the ranking still compares learning settings, not only algorithms. The thresholds (K, Q-statistic) come from the original works and were not tuned, so F1 partly reflects these choices. AUC gives a threshold-free view.

External validity. The extension uses only HDFS, where anomalies are rare. We did not use BGL there because about half of its windows are anomalous, which breaks the main assumption of unsupervised detectors. HDFS anomalies are often marked by specific templates, which may favour count-based models.

Conclusion validity. The random split may place later sessions in the training set (temporal leakage), as in the original study. Ten resamples give a limited picture of the variance.

Also, the Scott-Knott ESD groups depend on an effect-size threshold, not on a significance level.

V. Conclusion

We replicated part of the study of Wu et al. and extended it with two models that need fewer labels. Our four main findings are the following.

1) **Replication packages can change results.** Two issues in the original code (swapped precision and recall, truncated short sessions) help to explain two of our three differences with the paper. Running the original code next to a corrected version is what showed this.
2) **On HDFS, count features with labels give the best results.** A 10-tree Random Forest on message counts is the best and cheapest model, and it stays first when trained on 5,000 sessions. An LLM-based model is not always needed: LogGPT takes about 8,000 times the training time of the RF (GPU against CPU) and ranks third, although it does not need anomaly labels. Removing correlated features does not make the RF more accurate, but it helps to interpret the model.
3) **Without labels, the protocol matters as much as the model.** LogGPT's mean F1 moves by about 0.05 depending on whether K is 7 or 8, and the F1 of PCA on HDFS goes from 0.087 (LogGPT paper) to 0.487 or 0.768 (ours, mean over 10 resamples) depending on how it is trained. A fair comparison needs the same test sets, a fixed threshold rule, and repeated resamples.
4) **Explanations should be grouped and checked for stability.** Per-feature importance underrates correlated templates. Grouped SHAP values are consistent across methods and resamples, but the groups themselves change between resamples.

Future work could apply the extension to datasets where anomalies are not marked by specific templates. It could also study how to choose the LogGPT threshold using normal data only, and train LogGPT on more sessions.

VI. Team Contribution

The work was divided into three parts, one for each member.

- **Tran Vo** built the data pipeline (log parsing, partitioning, splitting and representations), carried out the correlation and redundancy analysis (step 3a), and implemented and evaluated the classical supervised models.
- **Pham Minh Dung Duong** implemented and evaluated the deep learning supervised models and compared our results with those of the original study (replication comparison).
- **Thanh Nghiep Quan** worked on the extension track: the two unsupervised and semi-supervised models, the Scott-Knott ESD ranking and the SHAP explanation.

All members took part in writing and reviewing the report.

References

[1] X. Wu, H. Li, and F. Khomh, "On the effectiveness of log representation for log-based anomaly detection," *Empirical Software Engineering*, 2023.

[2] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.

[3] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[4] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, 2023.

[5] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, 2009, pp. 117–132.

[6] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2007, pp. 575–584.

[7] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40.

[8] Q. Qin, R. Aghili, H. Li, and E. Merlo, "Preprocessing is all you need: Boosting the performance of log parsers with a general preprocessing framework," in *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2025.

[9] Z. Jiang, J. Liu, J. Huang, Y. Li, Y. Huo, J. Gu, Z. Chen, J. Zhu, and M. R. Lyu, "A large-scale evaluation for log parsing techniques: How far are we?" in *33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] X. Han, S. Yuan, and M. Trabelsi, "LogGPT: Log anomaly detection via GPT," in *IEEE International Conference on Big Data (BigData)*, 2023.

[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] J. E. Jackson and G. S. Mudholkar, "Control procedures for residuals associated with principal component analysis," *Technometrics*, vol. 21, no. 3, pp. 341–349, 1979.

[15] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[17] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," *IEEE Transactions on Software Engineering*, vol. 43, no. 1, pp. 1–18, 2017.

[18] C. Tantithamthavorn, "ScottKnottESD: The Scott-Knott effect size difference (ESD) test, version 3.0.0," https://github.com/klainfo/ScottKnottESD, r package.

[19] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.

[20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[21] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.